DEVOPS COHORT · LECTURE 1

Learning Go from Scratch

Your first step into the language that powers Kubernetes, Docker, Prometheus, and Terraform — the tools you'll use every single day as a DevOps engineer.

Duration	Level	Goal
1 Hour	Fresher / Beginner	Write & run your first Go program

1 Why Learn Go?

Before writing a single line of code, understand *why* this language matters for a DevOps engineer.

🧠 **Think of it this way:** You've probably written bash scripts — small programs that automate tasks. Go is what you graduate to when bash gets too messy, too slow, or too hard to maintain. It gives you the same power with proper structure, error handling, and the ability to build real tools.

🚀

Compiles to a Single Binary

Your whole app becomes one file. No Python venv, no Node modules, no runtime to install. Drop it anywhere and it just runs.

🧵

Built for Concurrency

Goroutines let you poll 1,000 servers for health status simultaneously — with just a few lines of code.

📦

Perfect for Containers

Since your app is one binary, Docker images stay tiny. Less to ship, less to worry about in production.

🌍

Powers the Tools You Use

Kubernetes, Docker, Prometheus, Terraform — all written in Go. Understanding Go means understanding the tools you work with every day.

💡 When you run `kubectl`, `terraform`, or `helm`, you are running a Go binary. Today you start writing tools like these.

2 Setting Up Your Environment

Step	What to do	Command to verify
Install Go	Download from **go.dev/dl**	`go version`
Code Editor	VS Code + Go extension	Open a .go file
First folder	Create `devops-bootcamp/`	`mkdir devops-bootcamp`

✅ **You're ready when:** running `go version` in your terminal prints something like `go version go1.22 linux/amd64`

3 Your First Go Program

Every Go program starts from this exact structure. Memorise it — you'll type it hundreds of times.

MAIN.GO

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello, DevOps Bootcamp!")
}
```

RUN IT TWO WAYS

```
# Run directly — for quick testing during development
go run main.go

# Build a real binary — this is what goes into Docker
go build -o hello
./hello
```

🔑 **3 things to always remember:** Every file starts with `package main`. Execution starts at `func main()`. You import packages with the `import` keyword.

Term	What it means
`package main`	This file is the entry point of a runnable program
`import "fmt"`	Bring in the "fmt" (format) package to print things
`func main()`	The first function that runs when the program starts
`go run`	Compile + run in one step (use during development)
`go build`	Compile into a binary file — this is what you ship

🐳 **DevOps connection:** The binary produced by `go build` is what goes inside your Docker image. Because it has no dependencies, your Dockerfile can be as simple as: copy binary → run binary. That's it.

4 Variables & Types

Go is **statically typed** — every variable has a fixed type. But it's smart enough to figure out the type most of the time.

```go
package main

import "fmt"

func main() {
    // Short declaration — Go figures out the type automatically
    name      := "api-server"     // string
    port      := 8080             // int
    isHealthy := true             // bool

    // Explicit type — when you want to be clear
    var timeout int    = 30
    var version string = "v1.0"

    // Constant — value never changes at runtime
    const maxRetries = 3

    fmt.Println(name, port, isHealthy, timeout, version, maxRetries)
}
```

`string` `int` `float64` `bool` `[]string (slice)` `map[string]int`

💡 The `:=` operator means *"create a new variable and assign this value."* Once a variable exists, use plain `=` to update it. You'll use `:=` constantly.

5 Functions

Functions in Go are straightforward — but they have one superpower: **multiple return values**. This is how Go handles errors.

BASIC FUNCTION

```go
func greet(name string) string {
    return "Hello, " + name
}
```

MULTIPLE RETURN VALUES — GO'S SUPERPOWER

```go
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("cannot divide by zero")
    }
    return a / b, nil  // nil means "no error"
}

func main() {
    result, err := divide(10, 2)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("Result:", result) // 5
}
```

⚠️ **Go has NO exceptions.** There is no try/catch. Errors are just values that functions return. The pattern `result, err := someFunc()` followed by `if err != nil` is something you will write hundreds of times. Get comfortable with it early — it makes your programs much more reliable.

6 Control Flow — if and for

Go only has `for`. There is no `while`. But `for` can do everything.

IF STATEMENT

```
if port > 1024 {
    fmt.Println("Valid user-space port")
} else {
    fmt.Println("Requires root privileges")
}
```

CLASSIC FOR LOOP

```
for i := 0; i < 5; i++ {
    fmt.Println(i)
}
```

FOR-RANGE — ITERATE OVER A SLICE (YOU'LL USE THIS THE MOST)

```
services := []string{"api", "worker", "cron"}

for i, svc := range services {
    fmt.Printf("[%d] Service: %s\n", i, svc)
}
```

FOR AS A WHILE LOOP (NO WHILE KEYWORD IN GO)

```
count := 0
for count < 3 {
    fmt.Println("Retrying connection...")
    count++
}
```

7 Structs — Grouping Related Data

Go has no classes. Instead, you use **structs** to group related fields together — like a form with named fields to fill in.

```go
package main

import "fmt"

// Define the struct — this is a blueprint, not a value yet
type Service struct {
    Name    string
    Port    int
    Healthy bool
}

func main() {
    // Create actual instances from the blueprint
    api := Service{
        Name:    "api",
        Port:    8080,
        Healthy: true,
    }

    fmt.Printf("Service %s is running on port %d\n", api.Name, api.Port)

    // A slice (list) of structs — very common pattern
    services := []Service{
        {Name: "api",    Port: 8080, Healthy: true},
        {Name: "worker", Port: 9000, Healthy: false},
        {Name: "cron",   Port: 9090, Healthy: true},
    }

    for _, svc := range services {
        status := "✅"
        if !svc.Healthy {
            status = "❌"
        }
        fmt.Printf("%s %-10s :%d\n", status, svc.Name, svc.Port)
    }
}
```

🎯 **This matters a lot:** HTTP handlers, database rows, API responses — everything in real Go programs is modelled as a struct. Once you understand structs, the rest of Go starts to click.

8 Hands-On Exercise

🏋️ Exercise: Service Health Printer

Write a Go program that:

- Creates a `Service` struct with **Name**, **Port**, and **Healthy** fields
- Creates a slice with **at least 3 services**
- Loops through the slice and prints each service in this format:

```
Service: api      | Port: 8080 | Status: Healthy
Service: worker   | Port: 9000 | Status: Unhealthy
Service: cron     | Port: 9090 | Status: Healthy
```

Hint: Use `fmt.Printf` and a `for range` loop. For the status, use an `if` statement.

---

9 What's Coming Next

1. **Go Basics (Today ✓)**
   Variables, functions, structs, control flow
2. **Packages, Modules & Interfaces**
   Organising code, go mod, writing testable code
3. **Goroutines & Channels**
   Go's concurrency model — poll 1,000 servers at once
4. **HTTP Server with net/http**
   Build your first web service from scratch
5. **Talking to PostgreSQL**
   Persistent storage, CRUD operations
6. **Dockerise the App**
   Build a tiny container image from your Go binary

10 Resources & Homework

go.dev/tour

🏆 Official interactive tour. Complete the **Basics** section as homework. Runs in your browser — no setup needed.

gobyexample.com

Bite-sized, runnable examples for every concept. Bookmark this — you'll use it daily.

pkg.go.dev

Official package documentation. Refer here whenever you use a new package.

roadmap.sh/golang

Visual learning roadmap. See what's ahead and track your progress.

📖 **Homework:** Complete the **Basics** section of the Go Tour at go.dev/tour/basics. It's interactive — you write and run code right in your browser. Takes about 30–45 minutes.

Prepared by **Shubham Rasal**